

1. Find the Majority Element (Boyer–Moore Voting Algorithm)

Question: Find the element that appears more than $n/2$ times in an array (the majority element). Explain the **Boyer–Moore Voting Algorithm** and its logic.

Simple Explanation: The Boyer–Moore Voting Algorithm is highly efficient because it finds the majority element in a single pass $O(n)$ time) and uses only $O(1)$ extra space. It works on the principle that if an element is truly the majority, it will always be the last one standing when paired up with all the other non-majority elements.

Algorithm Logic:

1. Initialize a candidate and a count to 0.
2. Iterate through the array:
 - If count is 0, set the current element as the new candidate.
 - If the current element matches the candidate, increment count.
 - If it doesn't match, decrement count.
3. The final candidate is guaranteed to be the majority element (assuming a majority element exists, as per the problem definition).

```
public class BoyerMooreMajorityFinder {
    /**
     * Finds the majority element in O(n) time and O(1) space.
     * Assumes a majority element (count > n/2) exists.
     */
    public static int findMajority(int[] nums) {
        int candidate = 0;
        int count = 0;

        for (int num : nums) {
            // If count is zero, nominate the current element as the new
            candidate
            if (count == 0) {
                candidate = num;
                count = 1;
            } else if (num == candidate) {
                // If it matches, strengthen the candidacy
                count++;
            } else {
                // If it doesn't match, cancel out one vote
                count--;
            }
        }

        // In a guaranteed scenario, the final candidate is the answer.
        return candidate;
    }
}
```

```
}
```

2. Find element appearing once where others appear twice (Optimized Bit Trick)

Question: Find the single element that appears exactly once in an array where all other elements appear exactly twice. Implement the solution using the most optimal **bitwise trick**.

Simple Explanation: The most optimized way is to use the **XOR (Exclusive OR)** bitwise operator. The property of XOR is that $A \oplus A = 0$ and $A \oplus 0 = A$. By XORing all the numbers in the array together, all pairs that appear twice cancel each other out to zero, leaving the single unique number as the final result.

```
public class UniqueElementFinder {  
    /**  
     * Finds the single unique element using the XOR operator in O(n) time  
     * and O(1) space.  
     * @param nums Array where every element appears twice, except for  
     * one.  
     * @return The element that appears only once.  
     */  
    public static int findSingle(int[] nums) {  
        int unique = 0; // The identity element for XOR is 0  
  
        // XOR all elements in the array  
        for (int num : nums) {  
            unique = unique ^ num;  
        }  
  
        // The final result holds the unique number  
        return unique;  
    }  
}
```